

This is unreleased documentation for Serverpod **Next** version.

For up-to-date documentation, see the [latest version](#) (4.0.0).

3..16.5

CUSTOM SERIALIZATION

 Copy as Markdown

Custom serialization

Custom serialization lets you pass your own hand-written Dart classes, such as value types from your codebase or classes from a third-party package, through endpoints and models. Serverpod can serialize any class that follows three rules:

1. The class must have a method called `toJson()` which returns a JSON serialization of the object.

```
Map<String, dynamic> toJson() {  
  return {  
    'name': 'John Doe',  
  };  
}
```

2. There must be a constructor or factory called `fromJson()`, which takes a JSON serialization as parameters.

```
factory ClassName.fromJson(  
  Map<String, dynamic> json,  
) {  
  return ClassName(  
    name: json['name'] as String,  
  );  
}
```

3. The class must be declared in the `config/generator.yaml` file in the server project. The path needs to be accessible from both the server package and the client package.

```
extraClasses:  
  - package:my_project_shared/my_project_shared.dart:ClassName
```



Ask AI

Using a custom class as a model field

If your custom class will be used as a field inside a generated `.spy.yaml` model, also implement a `copyWith()` method. The parent model's generated `copyWith` calls `.copyWith()` on each field to produce a deep copy. `copyWith()` is not needed when the class is only used as an endpoint parameter or return type.

```
ClassName copyWith({
  String? name,
}) {
  return ClassName(
    name: name ?? this.name,
  );
}
```

TIP

In the framework, `copyWith()` is implemented as a deep copy to ensure immutability. We recommend following this approach when implementing it for custom classes to avoid unintentional side effects caused by shared mutable references.

When a custom class is used as a field on a table model, it is stored as a `json` or `jsonb` column. See [Storing serializable fields as JSONB](#).

Where custom classes live

Custom classes must live where both the server and the client package can import them, typically a [shared package](#). See that page for creating one and wiring it into the server and client `pubspec.yaml` files. The same package can carry both generated models and hand-written custom classes. A plain Dart package works too, since `extraClasses` puts no Serverpod dependency requirements on the class.

A minimal custom class looks like this:

```
class ClassName {
  String name;
  ClassName(this.name);

  Map<String, dynamic> toJson() {
    return {
      'name': name,
    };
  }

  factory ClassName.fromJson(
    Map<String, dynamic> jsonSerialization,
  ) {
    return ClassName(
```

```

        jsonSerialization['name'],
    );
}
}

```

After declaring the class in `extraClasses`, save with `serverpod start` running, or run `serverpod generate`. The class can now be used in your endpoints with the full serialization and deserialization management that comes with Serverpod.

In your server project, you can create an endpoint returning your custom object.

```

import 'package:my_project_shared/my_project_shared.dart';
import 'package:serverpod/serverpod.dart';

class ExampleEndpoint extends Endpoint {
  Future<ClassName> getMyCustomClass(Session session) async {
    return ClassName(
      'John Doe',
    );
  }
}

```

Custom class with Freezed

Serverpod also has support for using custom classes created with the [Freezed](#) package.

```

import 'package:freezed_annotation/freezed_annotation.dart';

part 'freezed_custom_class.freezed.dart';
part 'freezed_custom_class.g.dart';

@freezed
abstract class FreezedCustomClass with _$FreezedCustomClass {
  const factory FreezedCustomClass({
    required String firstName,
    required String lastName,
    required int age,
  }) = _FreezedCustomClass;

  factory FreezedCustomClass.fromJson(
    Map<String, Object?> json,
  ) =>
    _FreezedCustomClassFromJson(json);
}

```

In the `config/generator.yaml`, you declare the package and the class:

```

extraClasses:

```

```
- package:my_shared_package/my_shared_package.dart:FreezedCustomClass
```

Custom class with ProtocolSerialization

If you need certain fields to be omitted when transmitting to the client-side, your server-side custom class should implement the `ProtocolSerialization` interface. This requires adding a method named `toJsonForProtocol()`. Serverpod will then use this method to serialize your object for protocol communication. If the class does not implement `ProtocolSerialization`, Serverpod defaults to using the `toJson()` method.

Implementation example

Here's how you can implement it:

```
class CustomClass implements ProtocolSerialization {
  final String? value;
  final String? serverSideValue;

  CustomClass({this.value, this.serverSideValue});

  // Serializes fields specifically for protocol communication
  Map<String, dynamic> toJsonForProtocol() {
    return {
      "value": value,
    };
  }

  // Serializes all fields, including those intended only for server-side use
  Map<String, dynamic> toJson() {
    return {
      "value": value,
      "serverSideValue": serverSideValue,
    };
  }
}
```

Client-side classes never need `toJsonForProtocol()`. Only the server uses it, to leave out fields that should not reach the client. The client always serializes with `toJson()`.